CSA07 Computer Networks — Assignment 1: Worked Answers

Smart Manufacturing Plant (CO1) — Tasks 1 to 6

Table of Contents

Task 1 — Size the Traffic Before Choosing Anything 1
Task 2 — Count the Domains in the Contractor's Drawing 3
Task 3 — Read a Captured Frame Layer by Layer 4
Task 4 — Choose a Medium for Each Run (There Is a Trap) 5
Task 5 — Wi-Fi 6 or 5G for Moving Equipment? 6
Task 6 — Interpret an Ethernet Port Report 7
Task 7 — Will the Receiver Notice the Error? 8
Task 8 — Correct the Error or Ask Again? 9
Task 9 — How Big Must the Window Be? 10
Task 10 — Find Where the Delay Comes From 11
Task 11 — Defend the Network Design 12
Task 12 — Re-evaluate and Adapt the Design (Decision: MODIFY) 12

GitHub repository: https://github.com/govukrithi-dev/Computer-networks.git

> **How to use this file.** These are worked answers and reference calculations. The assignment also asks for your own Packet Tracer files, Wireshark captures, screenshots and commit history. Run the programs in the accompanying code folder yourself, take your own screenshots, and commit one task at a time.

Task 1 — Size the Traffic Before Choosing Anything

Per-source traffic of one line (payload only)

Source	Working	Traffic
Robot controllers	40 × 200 B × 8 bit × (1000 / 2 ms = 500 per s)	40 × 0.8 Mbps = **32 Mbps**
Vibration sensors	600 × 50 B/s × 8	240 000 bit/s = **0.24 Mbps**
CCTV cameras	24 × 4 Mbps	**96 Mbps**
Office users	150 × 2 Mbps × 10 %	**30 Mbps**
One line	32 + 0.24 + 96 + 30	**158.24 Mbps**
Plant (8 lines)	8 × 158.24	**1 265.92 Mbps ≈ 1.27 Gbps**

Note: with Ethernet, IP and TCP headers the robot traffic on the wire is about 40 × 500 × 278 B × 8 = 44.5 Mbps per line, so the real figures are slightly higher.

Biggest sender and strictest delay. CCTV sends the most data (96 Mbps, 61 % of the line). Robot controllers have the strictest delay (10 ms deadline, one command every 2 ms). They are **not the same source**: the heaviest flow is the most delay-tolerant, the lightest-but-critical flow is the most delay-sensitive.

Is 1 Gbps enough?

- One line: 158.24 Mbps = 15.8 % of 1 Gbps — enough.
- Plant: 1 265.92 Mbps = **126.6 %** of 1 Gbps — **not enough**. The uplink is oversubscribed.
- If CCTV shares the same queue as robot commands, bursts of 1 500-byte video packets (12 µs each at 1 Gbps) sit in front of the 200-byte commands. Once the link is saturated the buffer fills (for example a 1 MB buffer = 8 ms of queue at 1 Gbps) and then drops packets. Robot commands then miss the 10 ms deadline or are lost. The fix is a separate queue with strict priority for robot traffic (QoS/VLAN), plus a 10 Gbps uplink.

Ranking by harm of a 100 ms delay

Rank	Traffic	Reason
1	Robot controllers	Deadline is 10 ms, so 100 ms is 10 times too late (50 commands missed). Risk to equipment and people.
2	CCTV (live)	Freezing and jitter in live monitoring and video analytics, but a buffer can hide part of it.
3	Office / ERP	Interactive, but 100 ms is hardly noticeable to a person.
4	Vibration sensors	One 50-byte reading per second per sensor; a 100 ms delay changes nothing for trend analysis.

Alternative — motion-triggered CCTV (5 % of the time)

- CCTV per line = 96 × 0.05 = 4.8 Mbps.
- Line = 32 + 0.24 + 4.8 + 30 = **67.04 Mbps**; plant = 8 × 67.04 = **536.32 Mbps = 53.6 %** of 1 Gbps.
- The 1 Gbps uplink is **no longer the bottleneck** (about 46 % headroom). Cost: local storage in the cameras or a local recorder, and delayed live view.

Delay versus volume (comparison)

Case	Plant load	Uplink use	Queue effect on robot commands
Continuous CCTV, shared queue	1 266 Mbps	126.6 %	Buffer full, drops, delay above 10 ms
Continuous CCTV, priority queue + 10 G	1 266 Mbps	12.7 % of 10 G	Commands served first, near-zero queue
Motion-triggered CCTV, 1 G	536 Mbps	53.6 %	Stable, but still needs priority for bursts

Task 2 — Count the Domains in the Contractor's Drawing

Marked drawing (text form)

```
 [8 PC]--Hub1--+
 [8 PC]--Hub2--+--- Switch1 (24-port) --- G0/0 [ROUTER] G0/1 --- Switch2 ---
12 IP phones
 [8 PC]--Hub3--+
   CD1,CD2,CD3        CD4 (sw-router)        CD5 (router-sw2)
CD6..CD17 (12 phones)
 |<--------------- Broadcast domain 1 --------------->|  |<-- Broadcast
domain 2 -->|
```

Collision domains = 17

- Each hub with its 8 PCs and the switch port it connects to is one shared segment: 3 domains.
- Switch1 to router G0/0: 1 domain.
- Router G0/1 to Switch2: 1 domain.
- Each of the 12 phone ports on Switch2: 12 domains (switch ports separate collision domains).
- Total = 3 + 1 + 1 + 12 = **17**.

Broadcast domains = 2. Hubs and switches forward broadcasts; only the router separates them. Domain 1 holds the three hubs, Switch1 and G0/0. Domain 2 holds G0/1, Switch2 and the phones.

Layer of each device

Device	OSI layer	Why
Hub	1 (Physical)	Repeats electrical signals to all ports, no addressing
Switch	2 (Data link)	Reads the MAC address and forwards per port
Router	3 (Network)	Reads the IP address and routes between networks
PCs and IP phones	1 to 7	End hosts run the whole stack

Why a hub cannot stop collisions. A hub does not look at frames and has no buffer. It copies every incoming bit stream to every other port, so two stations transmitting together still collide on the shared medium.

Point-to-point and multipoint. Switch1–router, router–Switch2, hub–Switch1 and every switch-port-to-phone link are point-to-point cables. Each hub with its PCs behaves as a multipoint shared segment (many stations on one collision domain).

Standards table

Technology	Standard	Defining body
Ethernet	IEEE 802.3	IEEE
Wi-Fi	IEEE 802.11	IEEE (Wi-Fi Alliance certifies products)

Technology	Standard	Defining body
IPv4	RFC 791	IETF
OSI model	ISO 7498	ISO
5G	3GPP releases; ITU IMT-2020	3GPP and ITU

Why buy only standard-compliant equipment. It guarantees interoperability between vendors, avoids lock-in, allows replacement and expansion without redesign, and gives predictable behaviour and safety/EMC compliance.

Alternative — three unmanaged switches instead of hubs

- Collision domains: 24 PC ports + 3 switch uplinks + 1 + 1 + 12 phones = **41**.
- Broadcast domains: still **2**.
- It fixes collisions, shared bandwidth (each PC gets a dedicated 100 Mbps/1 Gbps, full duplex).
- It does not fix the broadcast domain size, security or VLAN separation, QoS, monitoring, and loop protection is missing on many cheap unmanaged switches.

Task 3 — Read a Captured Frame Layer by Layer

Line	OSI layer	PDU
Ethernet II	2 Data link	Frame
IPv4	3 Network	Packet
TCP	4 Transport	Segment
Modbus write-register (12 bytes)	7 Application	Data (message)

Routers crossed. The sender sets TTL to 64 and the capture shows 61, so 64 − 61 = **3 routers**.

Bytes on the wire

- Frame = 14 (Ethernet) + 20 (IP) + 20 (TCP) + 12 (data) + 4 (FCS) = **70 bytes**.
- Overhead = 58 / 70 = **82.9 %**.
- 1 400-byte camera payload: 14 + 20 + 20 + 1 400 + 4 = 1 458 bytes; overhead = 58 / 1 458 = **3.98 %**.
- Small commands waste most of the frame on headers, so many small commands cost far more bandwidth per useful byte than a bulk video packet.

Fault to layer

Fault	Layer responsible
(a) Wrong MAC address	Data link (frame is not accepted by the NIC or the switch)
(b) TTL reaches 0	Network (router drops the packet and sends ICMP Time Exceeded)
(c) Destination port closed	Transport (TCP answers with RST)

TCP/IP model mapping. Application = OSI 5, 6, 7; Transport = OSI 4; Internet = OSI 3; Network access = OSI 1 and 2.

Complete the code (file `task03_layers.py`). The program output matches the expected output: 19, 25, 30 and 40 bytes, and overhead 28 bytes for 12 bytes of data. With a 1 400-byte message the same 28 bytes are only **2.0 %**, against **70 %** of the PDU for 12 bytes.

```
def send(message):
    pdu = message
    for layer, tag in LAYERS:
        head = "[" + tag + "|"
        tail = "|FCS]" if tag == "ETH" else "]"
        pdu = head + pdu + tail
        print(f"{layer} adds {tag} -> {len(pdu)} bytes")
    return pdu
```

Alternative — EtherType 0x8892 (no IP, no TCP)

- Headers saved: 20 + 20 = **40 bytes** per command.
- Ethernet requires at least 46 bytes of payload, so 12 data bytes are padded. The real minimum frame is 64 bytes, which is only 6 bytes less than the 70-byte frame.
- Commands per second on 100 Mbps (adding 20 bytes preamble and gap): with IP/TCP 100 Mbps / (90 × 8) = 138 889 per s; without, 100 Mbps / (84 × 8) = 148 810 per s. Gain is about **9 920 commands per s (+7.1 %)**.
- What is lost: routing between subnets (it stays in one broadcast domain), TCP reliability and ordering, port multiplexing, firewall/ACL and remote access from the head office, and easy use of standard IP tools.

Task 4 — Choose a Medium for Each Run (There Is a Trap)

Run	Medium	Deciding reason
(a) Robot cell, 85 m, arc welding	Cat6A S/FTP, grounded (or multimode fibre for full immunity)	Within the 90 m limit and shielded against welding noise; fibre is immune to EMI
(b) Office to shop floor, 140 m	Multimode fibre (or Cat6A with a switch at 70 m)	Over the 100 m copper limit
(c) Shop floor to data centre, 550 m	Single-mode fibre	OM3 multimode reaches only 300 m at 10 Gbps, so 550 m is the trap; single-mode gives 10 km and growth
(d) Plant to head office, 400 km	Leased line / carrier MPLS or DWDM	No plant-owned cable can span 400 km; the carrier regenerates the signal

Which run cannot use copper Ethernet at all. Run (d): 400 km is 4 000 times the 100 m limit. Runs (b) and (c) also exceed 100 m, but copper could be extended with intermediate switches; run (d) cannot, and even plain fibre needs carrier amplification.

Shannon capacity (250 MHz)

- SNR 40 dB = 10 000: C = 250 × 10^6 × log2(10 001) = **3 322 Mbps**.
- SNR 15 dB = 31.62: C = 250 × 10^6 × log2(32.62) = **1 257 Mbps**.
- Noise near the welders cuts the theoretical limit by about 62 %. The robot cell needs only 32 Mbps per line, so capacity is still enough, but the lower SNR raises the bit-

error rate and the number of retransmissions, which adds delay jitter against the 10 ms deadline. Shielding restores the SNR.

Why measured speed is below Shannon. Real modulation and coding are not ideal, headers and gaps use capacity, noise is not purely Gaussian (crosstalk, impulses), and a safety margin of SNR is kept.

Complete the code (`task04_shannon.py`). Output matches: 0.03 Mbps, 3 321.96 Mbps, 1 329.50 Mbps and 12.4 kbps.

```
def nyquist(bandwidth_hz, levels):
    return 2 * bandwidth_hz * math.log2(levels)

def shannon(bandwidth_hz, snr_db):
    snr = 10 ** (snr_db / 10)
    return bandwidth_hz * math.log2(1 + snr)
```

SNR for 56 kbps on 3 100 Hz. 2^(56 000 / 3 100) − 1 = 2.7 × 10^5, which is **54.4 dB**. A real analogue telephone line gives about 30 to 40 dB, so this is **not realistic**. 56k modems only worked because the downstream path was digital.

Alternative — Wi-Fi 6 in the welding area. 160 MHz at 15 dB: 160 × 10^6 × log2(32.62) = **804 Mbps**. Capacity is plenty, but a shared radio channel uses CSMA/CA, retries and interference, so there is no guaranteed 10 ms delivery. Wireless is **not acceptable** for hard real-time robot commands in the welding area; keep the shielded cable.

Task 5 — Wi-Fi 6 or 5G for Moving Equipment?

Hand-overs per trip (route 1.2 km, trip time 1 200 / 2 = 600 s)

Technology	Cells	Hand-overs	Time unreachable
Wi-Fi (30 m spacing)	1 200 / 30 = 40	39	39 × 150 ms = **5.85 s** (about 1 % of the trip)
Private 5G (300 m cells)	1 200 / 300 = 4	3	3 × 20 ms = **60 ms**

A Wi-Fi hand-over of 150 ms is 15 times the 10 ms deadline, and a 5G hand-over of 20 ms is still 2 times the deadline unless make-before-break (dual connectivity) is used.

5G service types

Traffic	Type
AGVs	URLLC (ultra-reliable low latency)
Cameras	eMBB (high bandwidth)
Sensors	mMTC (massive low-rate devices)

Where Bluetooth, Zigbee or LoRa win. Bluetooth Low Energy for tools, wearables, asset tags and headsets within a few metres; Zigbee for low-rate battery sensors in a mesh (lighting, temperature); LoRa for battery sensors over long distances and the outdoor yard. They give long battery life where Wi-Fi/5G would be wasteful.

Wi-Fi 6 with 802.11r. Fast transition typically brings hand-over to about 20 to 50 ms, which still misses 10 ms. What remains risky: roaming decided by the client, contention and retries, interference from welding, and unpredictable delay tails.

Alternative — LoRaWAN for 600 sensors. Demand = 600 × 50 B/s × 8 = 240 kbps against 5 kbps per gateway, which is **48 times** the capacity (about 48 gateways). Even one reading per minute gives 4 kbps (80 % of one gateway, and duty-cycle limits apply). Decision: continuously reporting vibration sensors stay on Wi-Fi (or wired/5G mMTC); slow temperature sensors can use LoRa.

Task 6 — Interpret an Ethernet Port Report

Meaning of the counters

- **Late collision:** a collision detected after the first 512 bits (64 bytes) of a frame. On a correct half-duplex segment it should not happen, so it indicates a duplex mismatch or an over-long segment.
- **CRC error:** the frame check sequence does not match, so the frame was corrupted or truncated.
- Together (412 late collisions, 1 907 CRC errors) they show the two ends disagree on duplex: the half-duplex end sees the full-duplex end transmitting and counts collisions, while the full-duplex end sees truncated frames.

Likely cause and fix. The PC is fixed at 100 Mb/s half duplex with auto-negotiation off while the gigabit switch port is on auto.

- Linux: `sudo ethtool -s eth0 autoneg on`, or force both ends: `sudo ethtool -s eth0 speed 100 duplex full autoneg off` and set the same on the switch.
- Windows: Device Manager → Network adapter → Advanced → Speed & Duplex → *Auto Negotiation*; PowerShell: `Set-NetAdapterAdvancedProperty -Name "Ethernet" -DisplayName "Speed & Duplex" -DisplayValue "Auto Negotiation"`.

Addresses

- `01:00:5e:0a:01:02`: the least significant bit of the first byte is 1, so it is a **group (multicast) address**; the prefix 01:00:5e is IPv4 multicast and the last 23 bits map from group address x.10.1.2.
- `3c:52:82:4e:11:07`: the first three bytes `3c:52:82` are the **OUI**, which identifies the manufacturer (look it up in the IEEE OUI registry); the first byte 0x3c means unicast and globally administered.

Jumbo frames (9 000 bytes). They help the CCTV storage server: efficiency of a TCP frame improves from 1 460 / 1 538 = 94.9 % to 8 960 / 9 038 = 99.1 %, with fewer frames and interrupts. Every NIC, switch port and VLAN in the path must be set to the same larger MTU, otherwise frames are dropped or fragmented.

Alternative — ring versus star of industrial switches

	Ring with fast recovery	Star of switches
Recovery after one cut	Tens of ms (ERPS about 50 ms, MRP 10 to 200 ms); RSTP about 1 to 2 s	No automatic recovery
Devices that lose connection	None (traffic goes the other way)	All devices on the switch whose uplink was cut, until repaired

Task 7 — Will the Receiver Notice the Error?

CRC of 1101011011 with generator 1011. Append three zeros: 1101011011**000**, then divide by 1011 (modulo 2, XOR).

```
1101011011000   (data + 000)
1011            XOR at bit 1
0110011011000   result
 1011           XOR at bit 2
0011111011000   result
  1011          XOR at bit 3
0001001011000   result
   1011         XOR at bit 4
0000010011000   result
     1011       XOR at bit 6
0000000101000   result
       1011     XOR at bit 8
0000000000100   result
remainder (last 3 bits) = 100
```

CRC bits = **100**, transmitted frame = **1101011011100** (confirmed by the program).

Received frame 1101011011101. Divide by 1011: the remainder is not zero, so the frame is **rejected** (it is the correct frame with the last bit flipped).

Burst errors and blind spots

- A generator of degree 3 is certain to detect every burst of length **up to 3 bits**.
- An undetectable pattern: any non-zero multiple of the generator, for example **0000000001011** (the generator itself). The program finds 1 023 such patterns for a 13-bit frame. Another one: two flipped bits 7 apart (0000010000001), because x^7 + 1 is divisible by x^3 + x + 1.

Two-dimensional parity (even), my 4 × 4 block

```
1 0 1 1 | 1
0 1 1 0 | 0
1 1 0 0 | 0
0 0 1 1 | 0
--------+--
0 0 1 0 | 1
```

A 4-bit error the method misses: flip the four corners of a rectangle, for example row 1 and row 2, column 1 and column 2. Every row and every column then has two flips, so all parities still match.

Internet checksum of 4500, 0073, 0000, 4000. Sum = 0x4500 + 0x0073 + 0x0000 + 0x4000 = 0x8573 (no carry). One's complement = 0xFFFF − 0x8573 = **0x7A8C**.

Complete the code (`task07_crc.py`). Output matches: transmitted 1101011011100, no error True, 1-bit error False, 3-bit burst False.

```
for i in range(len(dividend) - len(divisor) + 1):
    if rem[i] == "1":
        for j in range(len(divisor)):
            rem[i + j] = "0" if rem[i + j] == divisor[j] else "1"
```

Alternative — even parity versus CRC-3 (frame 1101011011100)

Error pattern	Even parity	CRC-3 (1011)
2-bit adjacent: 0000000000011	Missed (even number of flips)	Detected
2-bit, 7 apart: 0000010000001	Missed	**Missed**
4-bit: 0000000011101	Missed	**Missed** (it is a multiple of 1011)
4-bit burst: 0000000001111	Missed	Detected
3-bit: 0000000001011	**Detected** (odd number)	Missed

Parity catches every odd number of errors, including the 3-bit pattern the CRC-3 cannot see, but it misses every even number. CRC-3 detects all bursts up to 3 bits and most others, so it is much stronger overall.

Task 8 — Correct the Error or Ask Again?

- **Check bits for 16 data bits:** 2^r ≥ m + r + 1. For r = 4: 16 ≥ 21 fails; for r = 5: 32 ≥ 22 holds. So **5 check bits**; overhead = 5 / 16 = **31.25 %** (codeword = 21 bits).
- **Code {00000, 01011, 10101, 11110}:** distances are 3, 3, 4, 4, 3, 3, so **minimum distance = 3**. It detects d − 1 = **2 errors** and corrects (d − 1) / 2 = **1 error**.
- **Hamming(7,4), received 0110111:** s1 = c1⊕c3⊕c5⊕c7 = 0⊕1⊕1⊕1 = 1; s2 = c2⊕c3⊕c6⊕c7 = 0; s3 = c4⊕c5⊕c6⊕c7 = 1. Syndrome = 1 + 0 + 4 = **5**, so bit 5 is flipped. Corrected word **0110011**, data **1011**.
- **FEC or ARQ?** With a 40 ms retransmission time, FEC avoids the wait and the extra traffic and suits periodic sensor readings where fresh data matters. Video streaming and 5G use FEC because retransmitting costs a round trip that is too late for real-time use (5G also uses HARQ).

Complete the code (`task08_hamming.py`). Output: codeword 0110011; no error → syndrome 0, data 1011; error at 5 → syndrome 5, data 1011.

```
def decode(c):
    c = c[:]
    s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
```

```
s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
s3 = c[3] ^ c[4] ^ c[5] ^ c[6]
pos = s1 + 2 * s2 + 4 * s3
if pos:
    c[pos - 1] ^= 1
return pos, [c[2], c[4], c[5], c[6]]
```

Two flipped bits. Flipping positions 3 and 5 gives syndrome 6, so the decoder "corrects" bit 6 and delivers data 0101 instead of 1011. A single-error-correcting code cannot detect two errors.

Alternative — BER 1 in 1 000, 16-bit reading

	Hamming (21 bits)	Detect and retransmit (16 bits + detection code)
Bits sent per reading	21	16 / (1 − 0.01588) ≈ **16.26** on average (plus the detection bits and ACK)
P(reading in error)	2 or more errors: 2.07 × 10^{-4}	1 − 0.999^16 = 1.59 %
Average added delay	0 ms	0.0161 × 40 ms ≈ **0.65 ms**, but 1.6 % of readings wait 40 ms or more

FEC gives predictable delay at a cost of 31 % more bits; ARQ uses fewer bits but has delay spikes.

Task 9 — How Big Must the Window Be?

Timing. Frame = 1 500 × 8 = 12 000 bits.

- Transmission time = 12 000 / 10^9 = **12 µs**.
- Propagation time = 400 000 / 2 × 10^8 = **2 ms**.
- a = 2 ms / 12 µs = **166.67**.

Stop-and-wait. Utilisation = 1 / (1 + 2a) = 1 / 334.33 = **0.299 %**, which is **2.99 Mbps** on a 1 Gbps link.

Smallest window. W = 1 + 2a = 334.33, so **335 frames** (about 502 KB in flight, which equals the bandwidth-delay product).

- Go-Back-N needs a window of at most 2^k − 1: 2^8 − 1 = 255 is too small, 2^9 − 1 = 511 works, so **9 bits**.
- Selective Repeat needs a window of at most 2^(k−1): 2^9 = 512 ≥ 335, so **10 bits**.

Which resends fewer frames. Selective Repeat resends only the lost frame; Go-Back-N resends the lost frame and everything after it (up to 335 frames). The cost for Selective Repeat is a receiver buffer of about 335 frames (about 502 KB) and more complex logic.

Framing. HDLC marks the frame with the flag 01111110 and uses **bit stuffing** (insert a 0 after five consecutive 1s). PPP uses the flag 0x7E with **byte stuffing** (0x7D escape, then the byte XOR 0x20).

Complete the code (`task09_window.py`). Output matches the expected output (85.7 %, 2.3 %, 16.4 %, 100 %, 0.3 %, 2.3 %, 42.2 %).

```
a = prop / tx
return min(1, window / (1 + 2 * a)), a
```

Satellite link, 100 % utilisation. a = 150, so W = 1 + 2 × 150 = **301 frames**.

Alternative — Selective Repeat with window 64 on the head-office link. Utilisation = 64 / 334.33 = **19.1 %**, throughput = **191.4 Mbps** (instead of 1 Gbps). Receiver buffer = 64 × 1 500 = **96 000 bytes**; sequence numbers need at least 7 bits (2 × 64 = 128). It is cheaper but wastes about 80 % of the link.

Task 10 — Find Where the Delay Comes From

- **Expected propagation:** 400 000 m / 2 × 10^8 m/s = **2 ms one way, 4 ms round trip**. The last hop answers in about 12.6 ms, so propagation explains roughly **32 %** of the measured delay (the rest is queuing, processing and the provider's longer real route).
- **Hop 3, one value of 38.2 ms:** a one-off spike. The router answers ICMP probes on its slow control plane and may be busy, or a brief queue occurred. Later hops are fast (11.8 ms), so it is **not a fault**.
- **Hop 5, `* * *`:** the router does not reply to TTL-expired probes (ICMP filtered or rate-limited). Hop 6 answers, so traffic is passing through; **not a fault**.
- **Largest delay added:** between hop 3 (about 4.1 ms) and hop 4 (about 11.8 ms): **about 7.7 ms**. This is the long-distance provider leg.
- **Your own evidence (to be done by you):**

```
traceroute <server>          (Windows: tracert)
ping -c 20 <server>          (Windows: ping -n 20)
Wireshark capture filter: icmp or host <server>
```

In Wireshark, mark Ethernet II (L2), IPv4 with TTL (L3), ICMP/UDP/TCP (L4) and the application data, and keep the `.pcapng` file in the Task 10 folder.

Alternative — SNMP and NetFlow versus occasional ping/traceroute

Problem in the traceroute	SNMP counters	NetFlow	Occasional ping/traceroute
38.2 ms spike	Missed with 1 to 5 minute polling (shows queue drops only)	Missed (no delay measurement)	Seen only if the test runs at that moment
* * * hop	Not seen	Not seen	Seen, but must be interpreted (not a fault)
Sustained congestion or errors	Detected within one polling interval	Detected, with the top talkers	Detected only if the test runs then
Link failure	Immediate via SNMP trap	Flows stop	Detected only at the next test

Task 11 — Defend the Network Design

Area	Medium and technology	Ethernet and error/flow control, with numbers
Robot cells (welding)	Cat6A S/FTP, 85 m (Task 4), grounded	1000BASE-T, auto-negotiation on, full duplex (Task 6); CRC-32 detects bursts up to 32 bits (Task 7); priority queue for 32 Mbps of robot traffic (Task 1)
Office floor (140 m run)	Multimode fibre, Wi-Fi 6 for users	Office load 30 Mbps per line (Task 1)
Shop floor to data centre (550 m)	Single-mode fibre, 10 Gbps	Plant demand 1.27 Gbps is 126 % of a 1 Gbps link (Task 1); 10G gives 12.7 %
AGVs and moving equipment	Private 5G URLLC	3 hand-overs of 20 ms versus 39 of 150 ms for Wi-Fi (Task 5)
Sensors	mMTC or Wi-Fi; LoRa for slow sensors	240 kbps in total (Task 1); LoRa needs 48 gateways at 50 B/s (Task 5)
Plant to head office (400 km)	Leased line or MPLS, 1 Gbps	Round trip 4 ms; window of 335 frames (Task 9); Selective Repeat with 10-bit sequence numbers; TCP needs a window of at least 500 KB (64 KB gives only 131 Mbps)

Two biggest remaining risks

1. **AGV hand-over and radio reliability:** even 5G's 20 ms hand-over exceeds the 10 ms deadline, so make-before-break (dual connectivity), overlapping cells and a site survey are required.
2. **Single points of failure and oversubscription:** one WAN link and one uplink; Task 1 shows 1.27 Gbps against a 1 Gbps uplink. Fix with dual links, a 10G core and QoS.

Five plants. With continuous CCTV the head-office core sees 5 × 1.27 = 6.3 Gbps, so a 10 Gbps core and per-plant WAN links of at least 1.3 Gbps are needed; with motion-triggered CCTV it is 5 × 0.54 = 2.7 Gbps. Use a hierarchical design with local recording and a /21 per plant (1 500 devices). The design still works with these changes.

Alternative — whole plant on private 5G

Carry on 5G	Reason	Keep on cable	Reason
AGVs, mobile robots, handhelds	Need mobility; URLLC	Fixed robot controllers (32 Mbps per line, 256 Mbps for 8 lines)	Deterministic, immune to welding noise, cheap per Mbps
Sensors (1.9 Mbps for 8 lines)	mMTC suits many low-rate devices	CCTV (768 Mbps for 8 lines, 96 Mbps per line)	Too heavy for one cell, which shares its capacity
		Office (240 Mbps) and the backbone	Wi-Fi 6 and fibre are cheaper

Task 12 — Re-evaluate and Adapt the Design (Decision: MODIFY)

Assumption (state it in your report): each AGV sends a 200-byte command every 10 ms (0.16 Mbps), telemetry of 0.5 Mbps and one 4 Mbps navigation camera, so 4.66 Mbps per AGV.

A. Impact analysis

- AGV traffic: 50 × 4.66 = **233 Mbps**.
- Plant total: 1 265.92 + 233 = **1 499 Mbps ≈ 1.5 Gbps**, which is 150 % of a 1 Gbps uplink. The backbone must move to 10 Gbps fibre.
- Affected: wireless choice (Wi-Fi for AGVs is replaced by 5G URLLC), the backbone medium (1G to 10G), and error control (add HARQ and packet duplication on the radio; keep CRC-32 and TCP/UDP above).

B. Test the original design

- Per AGV on Wi-Fi: 39 hand-overs × 150 ms = **5.85 s** of lost reachability per 600 s trip. At one command per 10 ms that is **585 missed commands per trip** (0.975 %), so 50 AGVs miss 29 250 commands per trip cycle. The rule "must never miss a command" is **not met**.
- The 233 Mbps of AGV traffic also pushes the shared 1 Gbps uplink to 150 %.

C. Revised design

- Private 5G URLLC for AGVs and mobile equipment, with dual connectivity so hand-over takes about 0 ms (3 hand-overs of 20 ms = 60 ms without it).
- Fixed robots stay on shielded Cat6A; Wi-Fi 6 only for offices; CCTV on cable with motion-triggered recording; 10G single-mode fibre backbone in a ring; priority queue for robot and AGV traffic.

D. Defence with numbers and trade-offs

	Original (Wi-Fi AGVs)	**Modified (5G AGVs)**
Unreachable time per trip	5.85 s	0.06 s (or about 0 with dual connectivity)
Missed commands per trip	585	6 (or about 0)
Uplink load	150 % of 1G	15 % of 10G

Trade-offs: radio equipment, a private 5G core, spectrum and staff skills cost more, but the saving in risk (97 times less unreachable time) justifies it. A full redesign is unnecessary because the wired robot and office parts still meet their targets, so the decision is **MODIFY**.